Custom UDP Server vs. Godot ENet

Carlos Nunez

Mentor: DR. Girard

CSC498 — Senior Research Methods

Shippenburg University

Spring 2026

Abstract

This paper proposes a controlled comparison of two backend networking implementations for a Godot multiplayer game. ENet is built into the Godot Engine and a custom C++ UDP dedicated server. Both backends will share a single Godot client running in a peer-to-server topology. Performance will be secured by end-to-end round-trip latency and maximum concurrent client capacity before measurable degradation. Each backend will be tested at five client counts (1, 5, 10, 20, and 50). With three repetitions per count. A small game where the main objective is collecting an item, and it serves as a workload across both backends. The null hypothesis states that there will be no significant difference in performance at any client count. The alternative hypothesis predicts comparable performance at low counts, with a growing advantage for the C++ UDP server at higher counts.

1. Introduction and Problem Description

The Godot game engine ships with a built-in multiplayer API built on ENet, a UDP-based reliability library, and also exposes PacketPeerUDP for connecting clients to custom servers over raw UDP. ENet provides remote procedure calls, scene synchronization, automatic packet reliability, and connection management out of the box, but its dedicated server runs as a full Godot engine instance, carrying overhead that pure networking may not require. A custom C++ UDP server eliminates that overhead and gives the developer full control, at the cost of manually reimplementing what ENet provides by default.

The problem this paper addresses is the lack of a controlled comparison between these two backends in the context of a Godot client. Developers currently rely on assumptions, forum posts, and benchmarks from unrelated engines. No published study isolates how Godot's ENet dedicated server performs against a C++ UDP server when both serve the same Godot client under identical workloads.

This paper proposes producing that comparison through three different categories a custom C++ UDP server, a headless Godot ENet dedicated server, and a shared Godot client capable of connecting to either backend. A demo game, NodeStrike, will provide a consistent network workload, and a benchmarking tool will scale client counts in controlled increments.

Latency, packet throughput, and client capacity will be recorded and compared across both backends.

2. Literature Review

Multiplayer game networking is rarely visible to the player but plays a critical role in multiplayer communication. Multiplayer development is one of the more demanding areas of game development, requiring careful consideration of latency, scalability, and reliability. Two main options are available to developers building a multiplayer game with the Godot engine, either build in the ENet-based high-level networking system or a custom UDP server backend.

2.1. Godot Enet Stack

The Godot engine has a built-in high-level multiplayer API; it is built on top of ENet, a UDP-based networking library. Aukstakalnis describes the resulting ENet-based peer-to-server architecture as one in which “the game players interact with the client, while the server is a stripped-down version that updates the game to the clients” (Aukstakalnis 4). This is essentially a headless server, exported as a version of the Godot project that runs without audio or a graphical interface, leaving a process that runs game logic and relays state updates to clients. While this approach is simple and supports rapid early development, it later introduces architectural

constraints, most notably that the server runs as an instance of the engine itself, carrying overhead that may not be needed for pure networking tasks.

Aukstakalnis further notes that ENet provides convenient high-level abstractions such as remote procedure calls (RPCs), automatic packet reliability, and event-controlled callbacks such as on-connect and on-disconnect handlers. Based on the Godot Engine documentation on dedicated server exports, this headless mode is officially supported, but the developer is required to manage the game logic carefully to avoid client-only code from running on the server. To handle this, Godot supports flags and arguments such as --server, which are used to tell the code to run in server mode (Godot Foundation, “Exporting for Dedicated Servers”). The Godot Engine documentation also describes a different route for servers using lower-level networking with raw UDP, TCP, or HTTP for developers who want to implement their own (Godot Foundation, “UDPServer”).

A thesis from the DIVA portal examining server-client networking in Godot summarizes the tradeoff well, “Godot offers a high-level multiplayer API to facilitate the creation of multiplayer games in addition to supporting low-level networking via common protocols like UDP, TCP, HTTP, and SSL. Low-level networking is flexible and necessary for custom server implementations, but it usually takes much more time and knowledge to set up” (Aukstakalnis 6). This source provides the baseline that the proposed project aims to quantify, the convenience

of ENet versus the possible performance gains of a custom low-overhead UDP server written in the mid-level systems language C++.

2.2. Transport Protocol: TCP vs. UDP

The selection between the User Datagram Protocol (UDP) and the Transmission Control Protocol (TCP) is fundamental to any discussion of multiplayer networking performance. Al-Dhief et al. provide a comparison and conclude that UDP exhibits significantly lower latency than TCP due to its connectionless, no-order delivery model (Al-Dhief et al. 173). TCP carries built-in mechanics for packet retransmission, ordering, and congestion control, which retain data integrity and may be valuable for other applications but introduce significant delay in real-time-responsiveness applications.

In the context of multiplayer games, the reliability that TCP provides is largely unnecessary. Most updates to game state, such as player positions, input events, and world state, are superseded by the next update. A retransmitted packet from a previous frame has no value if it arrives late. UDP is therefore the better choice for the real-time gameplay loop, with sequence numbering layered on top by the application where order matters. Jones reinforces this rationale by stating that UDP's low overhead makes it an ideal choice for fast, real-time applications where latency matters more than guaranteed delivery (Jones).

2.3. Implementation Language for a Custom Server

There are many programming languages available when implementing a custom UDP server outside of the Godot Engine. Retunsky's benchmarking study of low-level I/O performance across C, C++, Rust, Go, Java, and Python finds that the results can be grouped into two clusters, C, C++, and Rust as high-performance languages and Rust, Go, Java, and Python as memory-safe languages (Retunsky). This classification emphasizes a fundamental trade-off between raw execution speed and the safety assurances offered by higher-level languages.

Arboleda, Arias, and Riveros add to this analysis by showing that C++ is materially more efficient than Python for parallel, I/O-bound workloads. This finding supports C++ as a strong choice for a high-performance game server expected to handle multiple clients simultaneously (Arboleda et al. 8). Zehra et al. reach a similar conclusion in a memory and time comparison between C++ and Python (Zehra et al.). There are also many available resources for learning how to set up a C++ UDP server. For example, GeeksforGeeks publishes a tutorial on UDP server-client implementation in C++ that demonstrates the basic socket workflow and indicates that setting up a server with C++ socket libraries is not as difficult as it might first appear, providing a useful comparison point against ENet implementations (GeeksforGeeks).

2.4. Network Topology

How the network is designed plays an important role in how it functions; network topology in a multiplayer game can impact performance characteristics and scalability. Çelik and

Seçinti examine peer-to-peer implementations and client-server topologies in the context of a multiplayer game. Their findings indicate that peer-to-peer implementations carry significant issues with synchronization, cheating, and latency, and that they constrain achievable client limits (Çelik and Seçinti 48). In a dedicated server topology, or client-server arrangement, all clients communicate independently with the server, which determines the game state. This arrangement is preferred for more competitive multiplayer experiences.

Ahde's thesis on real-time multiplayer server implementation provides a practical examination of dedicated server design, on one hand a server that handles the logic and processing, and on the other a server that handles the bandwidth and traffic management (Ahde 22). The proposed project adopts the dedicated-server topology in its simpler single-instance form, with one server process serving all connected clients.

2.5. Packet Loss, Buffer Bloat, and Heartbeat

Latency in a UDP server is not a single number but a distribution that depends on how the server's receive queue is managed under load. When packets arrive faster than the application can drain the socket buffer, queuing delay grows, and eventually new packets are dropped at the kernel level rather than enqueued. Industry analyses describe this progression through three regimes, idle, filling, and bloated, with median latency rising sharply as buffer occupancy approaches saturation (Raysync). Buffer bloat in particular has been identified as a primary contributor to latency-throughput trade-offs in real-time systems (Substack). Another concern is

detecting silent client disconnections; UDP's connectionless nature means the server cannot distinguish between a client that has stopped sending and one whose network has dropped. To address the challenge of silent disconnections, the standard architectural mitigation is the implementation of a heartbeat mechanism; this involves each client transmitting a small, periodic packet, allowing the server to evict any client from which no heartbeat has been received within a predefined, configurable timeout.

2.6. Summary

The literature reviewed in this paper establishes a foundation for the proposed project: a comparison of a custom C++ UDP server backend against Godot's built-in ENet dedicated server implementation. The existing body of work confirms that UDP is the appropriate transport protocol for real-time multiplayer games, that C++ offers a measurable performance advantage over higher-level languages for server-side I/O tasks, and that Godot's ENet system, while functional and convenient, carries architectural constraints inherent to its design as a stripped-down game engine instance. Together, these sources provide the theoretical grounding necessary to design, implement, and benchmark both server variants against consistent metrics of latency, packet handling, and client capacity.

3. Primary Objective

The primary objective of this project is to compare the performance of a custom C++ UDP dedicated server against Godot's built-in ENet dedicated server for a Godot multiplayer

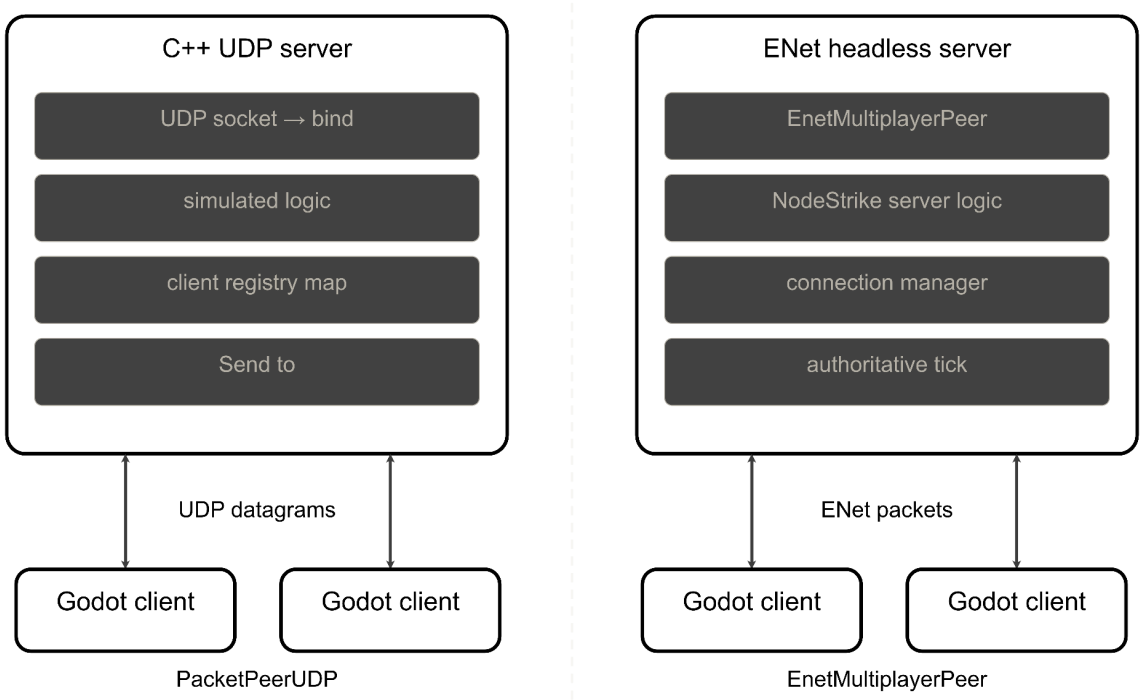
game, measuring end-to-end latency, packet-handling throughput, and maximum concurrent client capacity. The two backends will be evaluated using a single shared Godot client running in peer-to-server topology, executing identical NodeStrike workloads on the same host machine, with all measurements collected by the same instrumentation.

The total estimated time for the tests is 240 minutes, where each backend takes 120 minutes of testing. This is derived from five client-count steps multiplied by eight minutes per step multiplied by three repetitions per step.

4. Solution Description

4.1. System Architecture

The benchmark consists of three software components and one instrument layer. The first component is the shared Godot client, which implements both an ENet transport path using the built-in EnetMultiplayerPeer class and a raw UDP transport path using the PacketPeerUDP class. The client emits NodeStrike player input upstream and applies authoritative game state received downstream. The second component is the Godot ENet headless dedicated server, exported with the engine's official dedicated-server preset, running NodeStrike server logic and managing connections through ENet. The third component is the custom C++ UDP dedicated server, which binds a UDP socket on a configurable port, maintains a client registry indexed by IP and port, and broadcasts authoritative state to all registered clients on a fixed tick interval.



The instrumentation layer exists independently of the two backends under test. It is comprised of a network capture utility to monitor traffic on the host interface, a per-client benchmarking logger that records packet counts and high-resolution timestamps to a CSV file, and a tool that composes harnesses that scale the population of headless client containers in controlled increments.

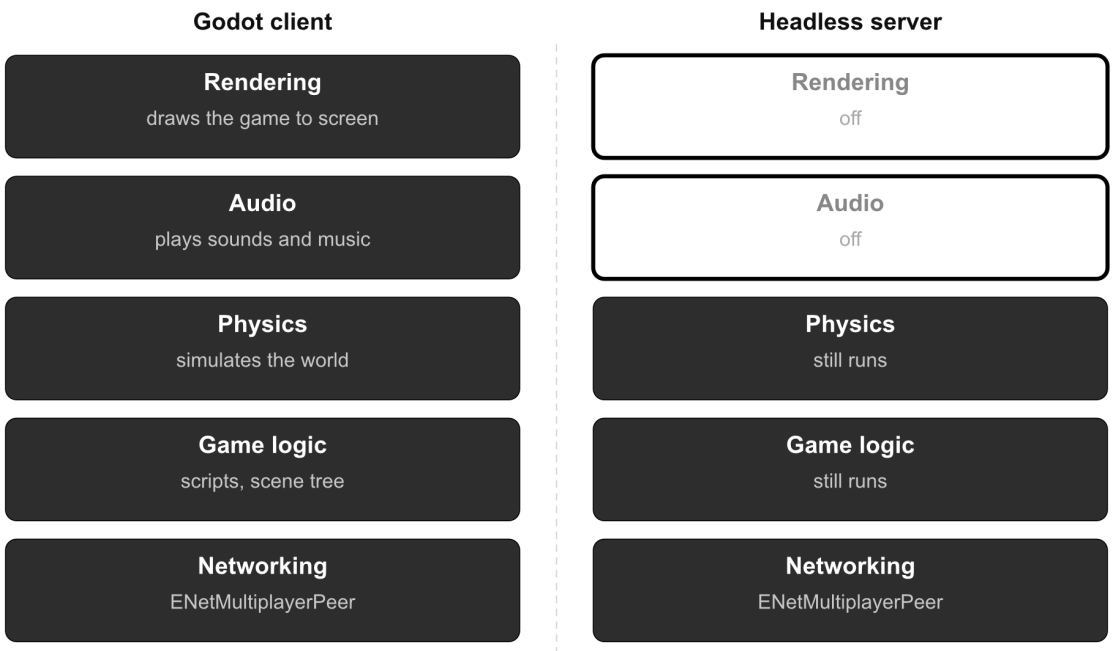
4.2. Class and component Inventory

Classes	Side	Purpose
ENetConnection	ENet backend	Low-level ENet host wrapper — stats, bandwidth, service loop
ENetMultiplayerPeer	ENet backend	A head-class ENet connection and multiplayer API
MultiplayerAPI	ENet backend	handles @rcp calls and syncs the scene
PacketPeerUDP	Shared client	Send and receive raw UDP packets from Godot client
socket / bind / recvfrom / sendto	C++ backend	POSIX socket calls used to create the UDP listener, accept packets, and broadcast state.
Client registry	C++ backend	In-memory map from client IP and port to PlayerState, used to track connected clients and their last heartbeat.

4.3. Headless Mode and Engine Overhead

Both servers run headless on the host machine. In a normal Godot client, the engine runs five subsystems, rendering, audio, physics, game logic, and networking. In the headless

dedicated-server export, rendering and audio are disabled, but physics, game logic, and networking continue to execute inside the engine process. The custom C++ UDP server runs only the networking layer and the minimum game logic required to maintain authoritative state. This difference in engine overhead is precisely what the benchmark is designed to characterize.



4.4. What is being built from scratch

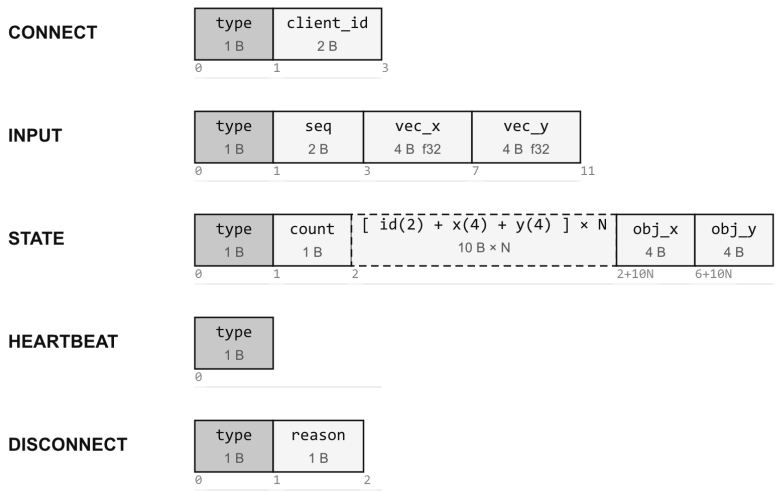
What is being made from scratch in this project:

- A C++ UDP server with a single-threaded receive loop, client registry, fixed-tick state broadcast, heartbeat-based eviction, and a binary packet format for player input and authoritative state.
- A tool that composes a benchmarking harness that scales headless Godot client instances in controlled increments and aggregates per-client metrics from each instance.
- A shared Godot client capable of switching between the ENet transport path and the raw UDP transport path with no changes to NodeStrike game logic.
- The NodeStrike demo game itself, including a grid map, an objective spawner, bot logic for client-side movement toward the nearest objective, and minimal client-side rendering for verification.

4.5. Packet Format

The custom C++ UDP service uses a format of how the packet will be parsed and delivered. Each packet begins with a one-byte message ID followed by a variable payload, depending on what type of data is being sent. These messages can be defined as CONNECT, sent by a client requesting to join; INPUT, sent by a connected client updating its current vector; and a sequence number, STATE, broadcasting a position of the current player and the objective. HEARTBEAT, sent by clients each second to maintain a connection to the server. And DISCONNECT, sent by the client requesting a disconnect. In contrast to this custom approach,

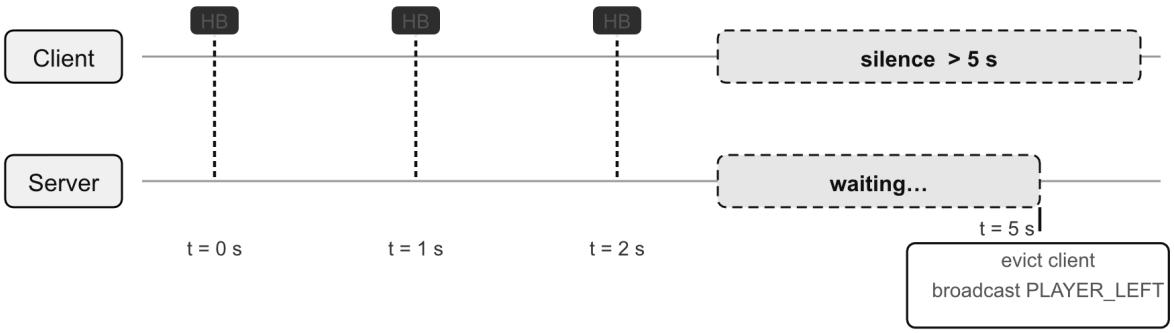
the ENet transport path utilizes Godot's native packet serialization. While this methodology diverges from the custom C++ implementation, it maintains a degree of functional equivalence sufficient for the comparative requirements of the benchmark.



4.6. Heartbeat and Client Eviction

To address the challenge of silent disconnections, each connection is maintained by a heartbeat packet transmitted to the server once per second. The server records a high-resolution timestamp for each client upon receipt. During each execution of the main loop, the server compares the current system time against the last-seen timestamp of every registered client, evicting any client whose latency window exceeds a predefined threshold. This timeout is set to

five seconds to maintain functional equivalence with the ENet server default, ensuring that connection management remains comparable across both backends.



4.7. The Node Strike Workload

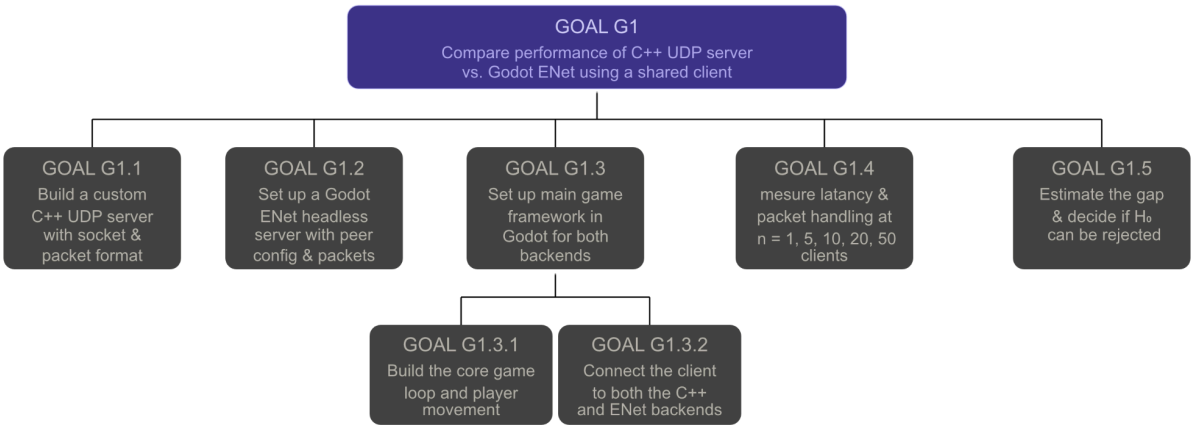
NodeStrike is a small objective-collection game intended to perform a consistent and easy-to-scale network workload. Players spawn on a grid map. An objective node spans a random cell. Each client computes the direction vector from its current position to the nearest objective and moves towards it: on contact, the objective is collected, and a new objective spawns in a new random cell. The game requires only two pieces of synchronized state per tick. The position of every player and the position of the current objective. Player input is restricted to a movement vector. This minimizes the complexity of the network behavior and simplifies game complexities so that it's easier to measure performance between backends rather than the game logic.

5. Hypotheses and Goal Tree

5.1. Hypothesis

Null hypothesis (H_0): the Godot ENet dedicated server and the custom C++ UDP server will show no statistically significant difference in any two of the secured metrics at any client count between two and fifty.

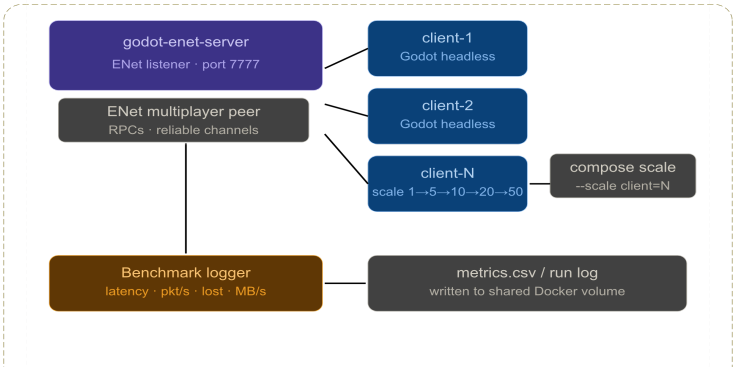
Alternative hypothesis (H_1): the godot ENet dedicated server will perform comparably to the C++ UDP server at low client counts (2 to 6), but the performance gap will grow significantly in favor of the C++ UDP server at higher client counts (30 to 50).

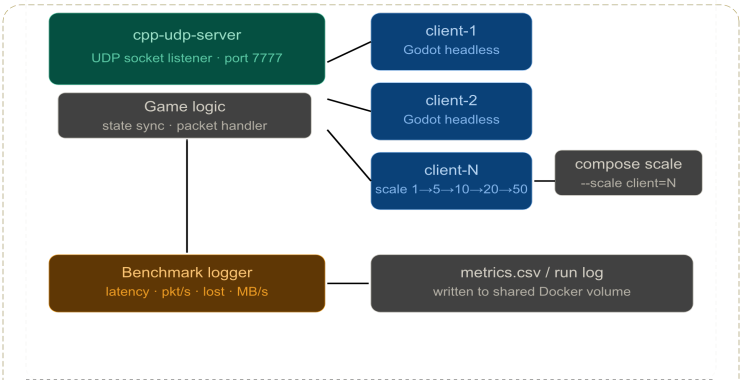


6. Experiment Design

6.1. Procedure

The same machine will run both backends at different times. After that, a script will start and set up a certain number of headless Godot clients, each of which will connect to the specified backend server. Wireshark records all UDP traffic on the host interface, and each client logs per-packet sent and received timestamps to a CSV file on a shared volume. The script measures traffic in each minute measurement window for each client count group. The harness terminates all client instances following each window. and initiates the next set of clients after restarting the backend for a new beginning.





6.2. Variable and control

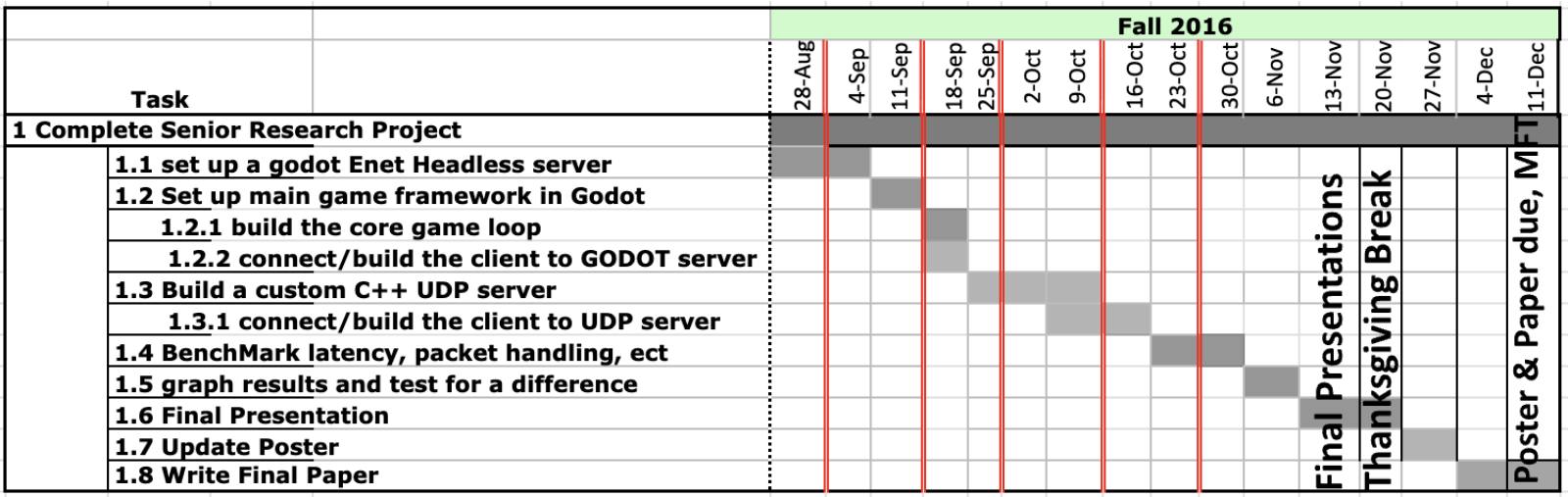
The independent variables are the choice of backend (ENet headless server or custom C++ UDP server) and the number of connected clients. The dependent variables are the latency

in milliseconds, the packet handling throughput in packets per second, the highest client counts sustained without degradation in bandwidth in megabytes per second, and the highest client count sustained without measurable degradation. Controlled variables include the host machine, the network interface, the operating system kernel parameters affecting socket buffer sizes, the Godot client build, the NodeStrike game logic and tick rate, and the testing time of day.

6.3. Data Analysis and Measurement

Each run's measurements are recorded in a CSV file, and latency distributions are calculated for each backend following each run. Packet handling throughput is calculated by dividing the measurement window by the total number of packets that Wireshark observes. The difference between client-side sent counts and server-side received counts, normalized by sent count, is used to calculate packet loss. Wireshark capture statistics are used to directly read bandwidth.

7. Schedule



- Task 1.1 — Set up the Godot ENet headless dedicated server with RPC and state synchronization. Tied to PR1.
- Task 1.2 — Set up the main game framework in Godot, including the NodeStrike core game loop (1.2.1) and the client connection path to the Godot server (1.2.2).
- Task 1.3 — Build the custom C++ UDP server with socket and packet format (1.3.1). Tied to PR2.
- Task 1.4 — Benchmark latency, packet handling, bandwidth, and client capacity. Tied to PR3 and PR4.
- Task 1.5 — Graph results and test for statistically significant differences between backends. Tied to PR4.
- Task 1.6 — Prepare and deliver the final presentation.
- Task 1.7 — Update the research poster with final results.
- Task 1.8 — Write the final paper.

8. Deliverables by Progress Report

- PR1 — Godot ENet headless server running with RPC and state synchronization working.
- PR2 — C++ UDP server running with socket, client registry, and state broadcast working.
- PR3 — Demo game complete; shared Godot client works with both C++ and ENet backends.
- PR4 — Experiment data collected; statistical analysis in progress.
- PR5 — Analysis complete.

9. Conclusion

This proposal describes a controlled comparison of the two backend networking implementations for a Godot multiplayer game. The games build in an engine, build in ENet dedicated server, and a custom C++ UDP dedicated server. The literature establishes UDP as the appropriate transport for real-time gameplay, C++ as a strong choice of implementation language for low-overhead server work, and the dedicated-server topology as the preferred arrangement for competitive multiplayer. The proposed benchmark addresses that gap with measurements of latency, packet handling throughput, and client capacity at five client counts, three repetitions per count, and a total testing budget of 240 minutes split evenly between backends.

References

Ahde, Jani. "Real-Time Unity Multiplayer Server Implementation." Bachelor's thesis, 2017.

Al-Dhief, Fahad Taha, et al. "Performance Comparison Between TCP and UDP Protocols in Different Simulation Scenarios." International Journal of Engineering & Technology, vol. 7, no. 4.36, 2018, pp. 172–176.

Arboleda, Francisco Javier Moreno, Mateo Rincon Arias, and Jesus Antonio Hernandez Riveros. "Performance of Parallelism in Python and C++." IAENG International Journal of Computer Science, vol. 50, no. 2, 2023, pp. 1–13.

Aukstakalnis, Lukas. "Server-Client Networking Implementation Using the Godot Game Engine." 2024.

Çelik, Arda Deniz, and Gökhan Seçinti. "Network Optimizing Software Solution for Multiplayer Gaming." ITU Journal of Wireless Communications and Cybersecurity, vol. 1, no. 1, 2024, pp. 47–56.

GeeksforGeeks. "UDP Server-Client Implementation in C++." GeeksforGeeks, 22 Mar. 2018, www.geeksforgeeks.org/cpp/udp-server-client-implementation-c/.

Godot Foundation. "Exporting for Dedicated Servers." Godot Engine Documentation, 2026, docs.godotengine.org/en/stable/tutorials/export/exporting_for_dedicated_servers.html. Accessed 27 Feb. 2026.

Godot Foundation. "UDPServer." Godot Engine Documentation, 2026, docs.godotengine.org/en/4.4/classes/class_udpserver.html. Accessed 27 Feb. 2026.

Jones, Charles. "Go UDP Programming: A Beginner-Friendly Guide to Building Fast, Real-Time Apps." DEV Community, 29 Oct. 2025, dev.to/jones_charles_ad50858dbc0/go-udp-programming-a-beginner-friendly-guide-to-building-fast-real-time-apps-4ik. Accessed 27 Feb. 2026.

“Kehom’s Forge.” Kehomsforge.com, 2024, kehomsforge.com/. Accessed 27 Feb. 2026.

Raysync. "Why Packet Loss Happens and How to Diagnose It." Raysync Blog, 2023.

Retunsky, Eugene. "Benchmarking Low-Level I/O: C, C++, Rust, Golang, Java, Python." Star Gazers, 26 Mar. 2023, medium.com/star-gazers/benchmarking-low-level-i-o-c-c-rust-golang-java-python-9a0d505f85f7.

"Socketserver — A Framework for Network Servers." Python Documentation, docs.python.org/3/library/socketserver.html.

Substack. "Buffer Bloat and the Latency-Throughput Trade-off." 2025.

Sudhi S. "Rust vs Go vs Python vs C++." Medium, 9 Apr. 2025, medium.com/@sudhis/rust-vs-go-vs-python-vs-c-afd7ae0aa5a7. Accessed 27 Feb. 2026.

Zehra, F., M. Javed, D. Khan, and M. Pasha. "Comparative Analysis of C++ and Python in Terms of Memory and Time." MDPI AG, 2020, doi.org/10.20944/preprints202012.0516.v1.